

Hierarchical JEPA-Flow Video World Model

Plain-English architecture brief for coder review

One-sentence proposition: train a video world model that first predicts the next abstract latent state, then predicts the next detailed latent state, and only after that trains a separate generator to render the next frame.

The goal is not to build a normal video diffusion model. The goal is to build a predictive latent hierarchy: the compressed state must carry the future-relevant structure, while the detailed latent carries texture and local visual information.

Key rule: the frame decoder is not allowed to teach the encoder during main training. The encoder is taught by future latent prediction against a stop-gradient EMA target branch.

1. The architecture in plain English

At time t , the model receives a short context clip, for example the last 4 video frames. The embedder turns that clip into detailed tokens called e_t . A bottleneck module then compresses e_t into a much smaller abstract state called c_t .

A slow-moving target copy of the encoder looks at the true future frame and produces target latents e_{plus} and c_{plus} . These target latents are stop-gradient: the model predicts them, but does not backpropagate into them.

A coarse flow model learns to generate c_{hat} from c_t . Then a fine flow model learns to generate e_{hat} from e_t plus a coarse future state. Finally, after latent training works, a cross-attention frame generator renders the predicted next frame from e_{hat} .

```
Data path
-----
context clip X_t  -> online encoder E      -> detailed latent e_t
                  e_t -> bottleneck B      -> abstract latent c_t

future frame y    -> EMA target encoder    -> stopgrad e_plus
                  e_plus -> EMA bottleneck -> stopgrad c_plus

Prediction path
-----
c_t               -> coarse flow F_c -> c_hat
(e_t, c_plus) early -> fine flow F_e  -> e_hat [teacher-forced]
(e_t, stopgrad(c_hat)) late -> fine flow F_e -> e_hat [real rollout]
stopgrad(e_hat)    -> frame generator -> x_hat
```

2. Recommended v0 dimensions

These dimensions are deliberately modest. They are large enough to test the idea on one H100, but small enough that debugging remains possible.

Component	Recommended v0	Reason
Input	4 context frames at 128 x 128; predict frame $t+1$ first	A single frame does not identify motion; short clips give velocity information.
Patch/tubelet size	1 x 16 x 16	Simple video-token setup: $4 \times 8 \times 8 = 256$ context tokens.
Online encoder E	VideoViT-small: depth 12, dim 384, 6 heads, MLP ratio 4	Transformer video encoders are the natural fit for latent video prediction.
Detailed latent e_t	256 tokens x 384 dim for context; 64 tokens x 384 dim for target	Enough detail for local appearance and motion without being huge.
Bottleneck B	ConvNeXt-style token mixer + 32 learned query slots	Forces compression from detailed perceptual state into abstract state.
Abstract latent c_t	32 tokens x 256 dim	About 8 percent of e_t scalar bandwidth; tight enough to prevent copying.
Coarse flow F_c	6 transformer blocks, dim 256, 8 heads	Predicts future abstract state c_{plus} from current c_t .

Component	Recommended v0	Reason
Fine flow F_e	8 transformer blocks, dim 384, 8 heads	Predicts future detailed latent e_plus from e_t and future coarse state.
Frame generator D	12-block cross-attention flow transformer in VAE latent space	Render visually plausible frames after the latent world model is working.

3. What teaches the encoder?

The encoder is taught by latent prediction. The online encoder must produce e_t and c_t that let the flow predictors recover the future target latents produced by the EMA target encoder.

The target encoder is not updated by backpropagation. It is a slow exponential-moving-average copy of the online encoder. This prevents the target from chasing the predictor every step and gives the online branch a stable future representation to predict.

```
EMA target update
-----
E_bar <- m * E_bar + (1 - m) * E
B_bar <- m * B_bar + (1 - m) * B

Recommended m schedule:
start 0.996, cosine-ramp toward 0.9999
```

4. Losses

4.1 Coarse JEPA-flow loss

The coarse flow learns to transform Gaussian noise into the future abstract latent c_{plus} , conditioned on c_t . This is a flow-matching loss, not plain endpoint regression.

```
Sample noise eps_c and time tau in [0, 1]
z_c = (1 - tau) * eps_c + tau * c_plus
u_c = c_plus - eps_c

L_c = mean_square( F_c(z_c, tau, c_t) - u_c )

Gradients from L_c update:
E, B, F_c

Gradients from L_c do NOT update:
E_bar, B_bar, c_plus
```

4.2 Fine JEPA-flow loss

The fine flow learns to transform Gaussian noise into the future detailed latent e_{plus} . Early training uses the true stopped c_{plus} as the coarse condition. Later training uses the predicted c_{hat} , but detached.

```
Early teacher-forced fine training
-----
c_condition = c_plus # stop-gradient target

z_e = (1 - tau) * eps_e + tau * e_plus
u_e = e_plus - eps_e

L_e = mean_square( F_e(z_e, tau, e_t, c_condition) - u_e )

Late predicted-coarse fine training
-----
c_condition = stopgrad(c_hat)

L_e = mean_square( F_e(z_e, tau, e_t, stopgrad(c_hat)) - u_e )

Critical rule:
L_e must not update F_c through c_hat.
```

4.3 Gaussian latent regularization

Use SIGReg-style Gaussian regularization on e_t and c_t . Do not use full VICReg as the mainline. Do not use no regularization. Tubelet dropping helps prevent shortcuts, but it does not reliably prevent latent collapse.

Reason: this model has flow predictors that start from Gaussian noise, so an approximately isotropic Gaussian latent distribution is a good match. It also gives a simple anti-collapse pressure without adding the extra variance/covariance tuning burden of VICReg.

```
Total latent objective
-----
L_latent = L_c + lambda_fine * L_e
          + lambda_e_reg * SIGReg(e)
          + lambda_c_reg * SIGReg(c)

Recommended defaults:
lambda_fine = 1.0
lambda_e_reg = 0.02
lambda_c_reg = 0.10
```

4.4 Frame generator loss

After latent training works, freeze the world model and train the frame generator. It should generate in a frozen image-VAE latent space, not raw RGB pixels. Raw pixel mean squared error should not be the main loss because it encourages blur.

```
Let A be a frozen image VAE encoder.
a_y = A(true_future_frame)

z_x = (1 - tau) * eps_x + tau * a_y
u_x = a_y - eps_x

L_frame = mean_square( D(z_x, tau, stopgrad(e_hat)) - u_x )

Gradients from L_frame update:
D only

Gradients from L_frame do NOT update:
E, B, F_c, F_e
```

5. Stop-gradient rules

Object	Stop-gradient?	Why
e_plus, c_plus	Always yes	They are targets from the EMA branch.
c_hat when fed into F_e	Yes during predicted-coarse training	Prevents L_e from corrupting the coarse predictor into a texture carrier.
e_hat when fed into D	Yes during frame-generator training	Frame generation should not rewrite the learned world model.
EMA target networks	No backprop ever	They update only through EMA.
Online encoder E and bottleneck B	No stopgrad on conditioning path during latent training	They must learn to preserve predictive information.

6. Training schedule

Stage	Train	Loss	Pass condition
0. Setup	Initialize $E_{bar} = E$ and $B_{bar} = B$	None	Data loader, shapes, EMA update, and mixed precision stable.
1. Coarse dynamics	E, B, F_c	$L_c + \text{SIGReg}$	c is noncollapsed; F_c beats copy and batch-mean baselines.

Stage	Train	Loss	Pass condition
2. Fine teacher forcing	E, B, F_c, F_e	L_c + L_e + SIGReg	F_e works when conditioned on true stopped c_plus.
3. Predicted-coarse fine training	E, B, F_c, F_e	L_c + L_e + SIGReg	Ramp c_condition from c_plus to stopgrad(c_hat); shuffled c_hat hurts.
4. Frame generator	D only; freeze E, B, F_c, F_e	L_frame	Frames depend strongly on e_hat; shuffled e_hat fails.
5. Optional polish	Usually D + maybe F_e only	Small LR, no encoder frame-loss updates	Only if all ablations pass.

7. Optimizer and runtime defaults

Setting	Recommended default
Optimizer	AdamW, betas (0.9, 0.95), weight decay 0.05
Gradient clipping	Global norm 1.0
Precision	bfloat16 or fp16 AMP
Batch	Global batch about 64 clips, using gradient accumulation if needed
Learning rate: encoder	2e-4 from scratch; 5e-5 to 1e-4 if pretrained
Learning rate: bottleneck	2e-4
Learning rate: flows	4e-4 for F_c and F_e
Learning rate: frame generator	2e-4
Schedule	10k warmup, cosine decay
Tubelet dropout	Drop about 40 percent of context tubelets; never drop target input
Condition dropout	About 10 percent for flow conditioning robustness

8. Dataset recommendation

Use Something-Something V2 for v0. It is action and object-interaction heavy, so temporal order matters. That makes it a better first stress test than a dataset where static appearance alone solves much of the task.

Practical setting: train at 128 x 128 first, predict t+1 first, then add horizons t+2 and t+4. Do not begin with long rollouts. First prove that the hierarchy works for one-step prediction.

The dataset has about 220k labeled short videos. At batch 64, 150k steps samples about 9.6M clips; because temporal crops and augmentations vary, this is reasonable for a v0 on a single H100 if the input pipeline is efficient.

9. Runtime tests and early-stop gates

Run these every 500 to 1,000 steps on a small fixed validation batch. They are designed to catch collapse, bypass, and numerical instability before wasting the run.

Test	What to measure	Stop or investigate if
Latent std	Mean/std per dimension for e and c	Many dimensions approach zero or flatline.
Effective rank	Covariance effective rank for e and c	Rank collapses sharply or keeps falling.
Coarse baseline	F_c error vs copy c_t and batch-mean baselines	F_c does not beat baselines after warmup.

Test	What to measure	Stop or investigate if
Shuffled c test	F_e with real c_hat vs shuffled/zeroed c_hat	Shuffled c performs almost as well as real c.
Teacher vs predicted gap	F_e with c_plus vs c_hat	Predicted-coarse never approaches teacher-forced performance.
Gradient health	Grad norm, NaNs, loss spikes	Repeated spikes, NaNs, or exploding EMA drift.
Decoder dependency	D with true/predicted/shuffled e_hat	Shuffled e_hat still gives plausible target frames.

10. What the coder should implement first

The first implementation should not include the frame generator. The first milestone is only the latent model: encoder, bottleneck, EMA branch, coarse flow, fine flow with teacher forcing, SIGReg, and the collapse/bypass tests.

Minimum milestone 1: a training script that runs Stage 1 and proves that c_t is noncollapsed and that F_c beats copy/batch-mean baselines.

Minimum milestone 2: add F_e with teacher-forced c_plus and prove that e_plus prediction depends on c_plus.

Minimum milestone 3: switch to stopgrad(c_hat), prove that shuffled c_hat hurts, then and only then add the frame generator.

11. Non-negotiable design constraints

Constraint	Reason
The target branch is always stop-gradient and EMA-updated.	Otherwise the target moves too fast and the representation can collapse or chase itself.
c_t must be much lower bandwidth than e_t.	Otherwise the abstract state becomes just a copy of the detailed latent.
F_e must be tested with shuffled/zeroed c.	This is the main test that the hierarchy is real and not decorative.
L_e must not backprop through c_hat into F_c.	Otherwise the fine loss can turn c into a texture/detail carrier.
The frame decoder is trained after latent learning and should not update the encoder.	Otherwise the system degenerates into a normal video generator instead of a JEPA-style latent world model.
Use Gaussian latent regularization plus tubelet dropout.	Regularization prevents collapse; tubelet dropout reduces shortcuts. They solve different problems.

12. Literature basis

This proposal combines established pieces rather than claiming that every piece is new. The distinctive design choice is the two-level coarse-to-fine latent transition with explicit bypass tests.

Idea used	Reference basis
Predicting representations instead of pixels	I-JEPA: Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture, arXiv:2301.08243.
Latent video prediction	V-JEPA: Latent Video Prediction for Visual Representation Learning, ICLR 2024 / OpenReview.
Flow matching as fast generative transition	Flow Matching for Generative Modeling, arXiv:2210.02747.
Latent-space visual generation and cross-attention conditioning	High-Resolution Image Synthesis with Latent Diffusion Models, arXiv:2112.10752.
Gaussian regularization for JEPA collapse prevention	LeWorldModel / LeJEPA line: stable JEPA from pixels with SIGReg-style Gaussian latent regularization.
Dataset	Something-Something V2 dataset card / dataset release: about 220,847 human-object interaction videos.